10 Tail Recursion and Loops

Instructor: Lin Hong

A323 ✆ Ext. 6294 ✉ linhong@mpu.edu.mo

https://canvas.mpu.edu.mo/courses/4091/

B.Sc. in Computing, Faculty of Applied Sciences, Macao Polytechnic University

23 September 2025



Outline

1. **Some Recursive Problems**
   - Computing Integer Square Roots
   - Permutation of List Elements
   - Some Recursive Mathematical Functions
2. **Tail Recursion**
   - Fibonacci Sequence
   - Converting Tail Recursions to Loops
   - Factorial
   - Integer Powers
   - Polynomial Evaluation — Horner's Rule



Textbook §4.2, §4.4 – 4.7.

Computing Integer Square Roots

- Given an integer $x \geqslant 0$, the integer square root $int_sqrt(x)$ returns an integer r such that
$$r^2 \leqslant x \leqslant (r+1)^2 - 1.$$
- Obviously, $int_sqrt(x) = \lfloor \sqrt{x} \rfloor$.
- Since this is a problem related to multiplication, we try to reduce the problem to solving the integer square root of $y = \lfloor \frac{x}{4} \rfloor$. Thus, we have $4y \leqslant x \leqslant 4y+3$.
- Let $s = int_sqrt(y)$, that is, $s^2 \leqslant y \leqslant (s+1)^2 - 1$, we have
$$(2s)^2 = 4s^2 \leqslant 4y \leqslant x \leqslant 4y+3 \leqslant 4(s+1)^2 - 4 + 3 = (2s+2)^2 - 1.$$
- Therefore, the integer square root of x can be computed as
$$int_sqrt(x) = \begin{cases} 0 & \text{if } x = 0, \\ 2s+1 & \text{otherwise, if } (2s+1)^2 \leqslant x, \\ 2s & \text{otherwise.} \end{cases}$$

Computing Integer Square Roots — Code

```
1  def int_sqrt(x):
2      if x == 0:
3          return 0
4      else:
5          r = 2*int_sqrt(x//4)+1
6          return r if r*r <= x else r-1
```

The recursive calls for computing the integer square roots of 100, 900 and 4000 are expanded respectively below.

$$\begin{array}{c} 100 \Rightarrow 25 \Rightarrow 6 \Rightarrow 1 \Rightarrow 0 \\ 11, \boxed{10} \Leftarrow \boxed{5}, 4 \Leftarrow 3, \boxed{2} \Leftarrow \boxed{1}, 0 \Leftarrow 0 \end{array} \qquad \begin{array}{c} 900 \Rightarrow 225 \Rightarrow 56 \Rightarrow 14 \Rightarrow 3 \Rightarrow 0 \\ 31, \boxed{30} \Leftarrow \boxed{15}, 14 \Leftarrow \boxed{7}, 6 \Leftarrow \boxed{3}, 2 \Leftarrow \boxed{1}, 0 \Leftarrow 0 \end{array}$$

$$\begin{array}{c} 4000 \Rightarrow 1000 \Rightarrow 250 \Rightarrow 62 \Rightarrow 15 \Rightarrow 3 \Rightarrow 0 \\ \boxed{63}, 62 \Leftarrow \boxed{31}, 30 \Leftarrow \boxed{15}, 14 \Leftarrow \boxed{7}, 6 \Leftarrow \boxed{3}, 2 \Leftarrow \boxed{1}, 0 \Leftarrow 0 \end{array}$$

Permutation of List Elements

- How to generate the permutations of r elements from the n elements stored in a list s?
  - Place s_0 at the head position, followed by the permutations of $r-1$ elements from the remaining $n-1$ elements $s_1, s_2, \ldots, s_{n-1}$;
  - Place s_1 at the head position, followed by the permutations of $r-1$ elements from the remaining $n-1$ elements $s_0, s_2, \ldots, s_{n-1}$;
  - $\vdots$
  - Place s_{n-1} at the head position, followed by the permutations of $r-1$ elements from the remaining $n-1$ elements $s_0, s_1, \ldots, s_{n-2}$.
- The base cases are obvious.
  - There's no permutation when $n < r$.
  - There's only one empty permutation when $n \geqslant r = 0$.
- How many permutations all together?

$$P(n,r) = \begin{cases} 1 & \text{if } n \geqslant r = 0, \\ n \times P(n-1, r-1) & \text{if } n \geqslant r \geqslant 1. \end{cases}$$

Permutation of List Elements — Code

- We yield all the permutations from s as a sequence of lists, each of length r.
- We *choose* the head element first, $s[i]$, then *reduce* the problem to the permutation of the remaining elements, $s[:i]+s[i+1:]$, and *combine* each t in the results of the sub-permutation with the head element, $[s[i], *t]$, as the final result.

```
1 def permutations(s, r):
2     if r == 0:
3         yield []
4     else:
5         for i in range(len(s)):
6             yield from ([s[i], *t] for t in permutations(s[:i]+s[i+1:], r-1))
```

Some Recursive Mathematical Functions

Factorial

$$n! = \begin{cases} 1 & \text{if } n = 0, \\ n \times (n-1)! & \text{if } n \geqslant 1. \end{cases}$$

Fibonacci numbers

$$0, 1, 1, 2, 3, 5, 8, 13, \ldots$$

$$fib(n) = \begin{cases} 0 & \text{if } n = 0, \\ 1 & \text{if } n = 1, \\ fib(n-1) + fib(n-2) & \text{if } n \geqslant 2. \end{cases}$$

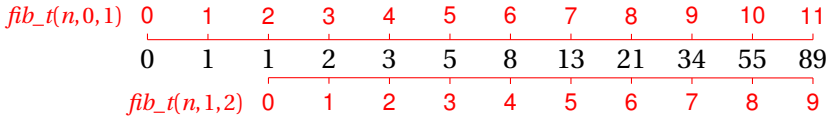
Going upstairs: you can choose to step either one or two stairs at a time, how many ways to go up n-stairs? For example, to go up 4 stairs, you have 5 ways:

$$1 \to 1 \to 1 \to 1 \qquad 1 \to 1 \to 2 \qquad 1 \to 2 \to 1 \qquad 2 \to 1 \to 1 \qquad 2 \to 2.$$

Tail Recursion

- If we don't fix the first two items of the Fibonacci sequence, instead, we specify them as parameters a and b, then the Fibonacci numbers can be defined as $fib(n) = fib_t(n, 0, 1)$, where

$$fib_t(n, a, b) = \begin{cases} a & \text{if } n = 0, \\ b & \text{if } n = 1, \\ fib_t(n-2, a+b, b+(a+b)) & \text{if } n \geqslant 2. \end{cases}$$



- This is a tail recursion, where every recursive call is the last call before return.
- A tail recursion can be transformed to a loop directly.

Converting Tail Recursions to Loops

- The loop condition is the condition when you do recursive calls.
- The loop body consists of the statements under the condition.
- Each tail recursive call is replaced by an iteration step, where *the function formal paramenters* are assigned with *the actual arguments in the call.*
- The base cases are placed after the loop.

```
1  def fib_t(n, a, b):
2      # fib_t(n, a, b) = fib_t(n−2, a+b, b+(a+b))   for n ⩾ 2.
3
4
5      while n >= 2:
6          n, a, b = n−2, a+b, b+(a+b)
7      return a if n == 0 else b
```



Converting Tail Recursions to Loops — Factorial

- The factorial function can also be defined as a tail recursion with an additional *accumulator parameter* p:

$$fact_t(n,p) = p \times n! \begin{cases} p & \text{if } n = 0, \\ p \times (n \times (n-1)!) = (p \times n) \times (n-1)! & \\ \qquad = fact_t(n-1, p \times n) & \text{if } n \geqslant 1. \end{cases}$$

- We transform the function definition into the following loop:

```
1 def fact_t(n, p):
2     while n > 0:
3         p, n = p*n, n-1
4     return p
```

Computing Integer Powers

- A naïve algorithm to compute integer powers of a number x would be:

$$pow_na(x,n,p) = px^n = \begin{cases} p & \text{if } n = 0, \\ (px)x^{n-1} = pow_na(x,n-1,px) & \text{if } n \geqslant 1. \end{cases}$$

- A faster reduction can be achieved by dividing the exponent in half:

$$pow_sq(x,n,p) = px^n = \begin{cases} p & \text{if } n = 0, \\ p(x^2)^k = pow_sq(x^2,k,p) & \text{if } n = 2k \geqslant 2, \\ (px)(x^2)^k = pow_sq(x^2,k,px) & \text{if } n = 2k+1 \geqslant 1. \end{cases}$$

- We convert the tail recursion into a loop:

```
1 def pow_sq(x, n, p):
2     while n > 0:
3         x, n, p = x*x, n//2, p if n%2 == 0 else p*x
4     return p
```

Polynomial Evaluation — Horner's Rule

- To evaluate a polynomial $a_0x^0 + a_1x^1 + \cdots + a_{n-1}x^{n-1}$ by tail recursion, we must add a whole term sx^n to obtain the accumulator!

$$\left(a_0x^0 + \cdots + a_{n-2}x^{n-2} + a_{n-1}x^{n-1}\right) + sx^n = \left(a_0x^0 + \cdots + a_{n-2}x^{n-2}\right) + (a_{n-1} + sx)\,x^{n-1} \quad (n \geqslant 1).$$

- Thus, the tail recursive function is,

$$f(x, a, n, s) = \begin{cases} sx^0 = s & \text{if } n = 0, \\ f(x, a, n-1, a_{n-1} + sx) & \text{if } n \geqslant 1. \end{cases}$$

- We transform the function definition into the following loop (Horner's rule):

```
1  def f(x, a):
2      s = 0
3      for n in range(len(a), 0, -1):
4          s = a[n-1] + s*x
5      return s
```

